

SISTEMAS DISTRIBUIDOS

CLASE PRÁCTICA - REPLICACIÓN Y CONSISTENCIA



Hoja de ruta

1. Bajar la app ntfy.sh
2. Ejemplos de replicación sincrónica y asincrónica en PostgreSQL
3. Live coding
4. <más cosas, pero no puedo spoilear>
5. <más cosas, pero no puedo spoilear>

REPLICACIÓN ASINCRÓNICA CON PostgreSQL

REPLICACIÓN SINCRÓNICA CON PostgreSQL

LIVE CODING

Caso de uso

- Los usuarios pueden realizar una compra usando un endpoint HTTP:
 - POST {userId, productId, comment}
- Por cada compra hay que bajar el stock del producto

Caso de uso

- Por cada compra hay que mandar una push notification al usuario

PUSH NOTIFICATIONS APP



ntfy.sh

subscription:
ta050

Caso de uso

- El servidor está MUY flojo de papeles, se reinicia seguido de forma inesperada
 - No crean que en el cloud los nodos viven para siempre (ej: AWS Spot instances son baratas pero te las pueden apagar cuando quieran! Al menos te avisan 2 min antes)
- Los usuarios compran desde una mobile app, la conexión puede ser intermitente

Dual writes

Un **contexto** de ejecución **sin estado durable**,

que **escribe en múltiples sistemas** que **no comparten una transacción**,

no puede garantizar que ambas escrituras ocurran.

Este sistema **no es eventualmente consistente** porque las inconsistencias no van a resolverse automáticamente. En muy poco tiempo se vuelve un **sistema inconsistente**.

Ejemplos de contextos de ejecución

No durable: HTTP endpoint handler, background thread, subscriber de pub/sub (si no estoy suscrito cuando se emite un mensaje lo pierdo), etc. Si hay algún error **no puedo asegurar que voy a intentarlo nuevamente**.

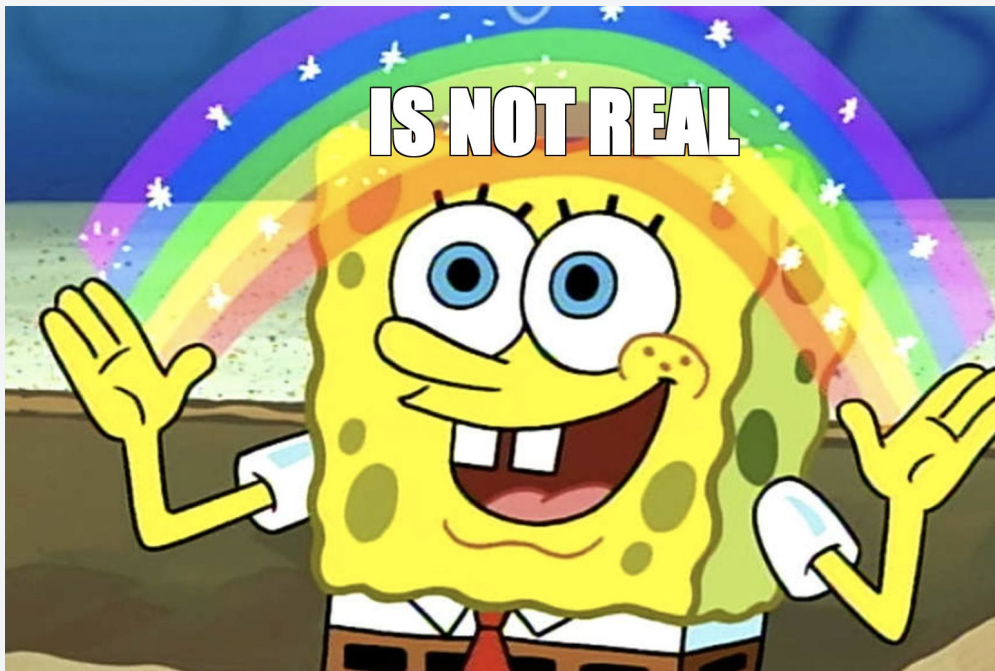
Durable: Consumidor de una queue o stream (SQS, RabbitMQ, Kafka, etc.), workflow orchestration (Temporal.io, AWS step functions, etc.), consumir el WAL de la base de datos, etc. **Si hay algún error puedo volver a intentarlo** (tal vez procesando nuevamente algunos mensajes).

Caso de uso

- No quiero que por problemas de red un usuario duplique su compra
- No quiero enviar infinitos push notifications a los usuarios si tengo un error en mi procesador de eventos
 - Por experiencia propia a los usuarios no les suele gustar mucho

Delivery semantics

- **at-most-once**: puedo perder datos pero nunca duplicarlos
- **at-least-once**: puedo duplicar datos pero nunca perderlos
- **exactly-once**:



Referencias

- Repo de la clase: <https://github.com/fiubaTA050/replication-consistency>
- <https://www.confluent.io/blog/dual-write-problem/>
- <https://youtu.be/rZPRjLMWOao?t=2683>: si enviamos un mensaje a otro sistema pero no recibimos una respuesta, no podemos saber qué pasó.
- <https://bravenewgeek.com/you-cannot-have-exactly-once-delivery/>
- <https://blog.bulloak.io/post/20200917-the-impossibility-of-exactly-once/>
- <https://www.confluent.io/blog/exactly-once-semantics-are-possible-heres-how-apache-kafka-does-it/>
- https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_TransactWriteItems.html
- <https://stripe.com/blog/idempotency>